
Instructor: Dr. V. Prakash Singh

Email: vpspeaks@gmail.com

Learning Objectives

By the end of this lab you will be able to:

- Formulate a real signal-processing problem (baseline wander removal) as a supervised regression problem.
- Implement linear and polynomial regression, with and without ℓ_2 (ridge) and ℓ_1 (lasso) regularization, **from first principles** (closed-form / gradient descent), before cross-checking against `scikit-learn`.
- Reason quantitatively about the bias–variance trade-off using model order and regularization strength as knobs.
- Design and justify an evaluation protocol for a signal-restoration task, going beyond a single scalar metric.
- Package a reproducible experiment as a version-controlled repository, suitable for submission and for a portfolio.

Background

The electrocardiogram (ECG) is contaminated by many artifacts, one of the most common of which is *baseline wander* – a slow, low-frequency drift (typically below 0.5–1 Hz) of the isoelectric line, caused by patient respiration, body movement, or poor electrode contact. Baseline wander does not destroy diagnostic information the way high-frequency noise does, but it corrupts amplitude-based measurements (e.g. ST-segment elevation, used to diagnose ischemia) and must be removed before automated analysis.

A clean way to think about baseline wander is as an additive model:

$$x[n] = s[n] + b[n], \quad n = 0, 1, \dots, N-1 \quad (1)$$

where $x[n]$ is the observed (corrupted) ECG, $s[n]$ is the clean ECG we ultimately care about, and $b[n]$ is the slowly-varying baseline drift. Because $b[n]$ is smooth and low-frequency while $s[n]$ contains sharp, high-frequency QRS complexes, we can estimate $\hat{b}[n]$ using **regression**: fit a low-order

function of time (or of a small set of “anchor” samples known to lie on the isoelectric line) to the signal, and treat that fit as an estimate of the baseline. The corrected signal is then $\hat{s}[n] = x[n] - \hat{b}[n]$.

This is precisely a regression problem: the input is time (or an engineered time-like feature), the target is the (unknown) baseline value, and the model complexity (polynomial degree) and regularization strength directly control whether the fit tracks only the slow drift or starts leaking into the QRS complexes themselves – a very visual, very concrete instance of overfitting.

Dataset

You will work with real ECG recordings from the **MIT-BIH Arrhythmia Database**, freely available on PhysioNet (<https://physionet.org/content/mitdb/1.0.0/>), accessible in Python via the `wfdb` package:

```
pip install wfdb neurokit2 numpy scipy scikit-learn matplotlib
```

```
import wfdb
record = wfdb.rdrecord('100', pn_dir='mitdb') # downloads record 100
ecg = record.p_signal[:, 0]                  # first channel, in mV
fs = record.fs                               # sampling frequency (360 Hz)
```

Since MIT-BIH records are relatively clean, you will also **synthetically add baseline wander** of known shape (so that you have ground truth to evaluate against) using a sum of low-frequency sinusoids and/or a slow polynomial drift, e.g.

$$b[n] = A_1 \sin(2\pi f_1 n / f_s + \phi_1) + A_2 \sin(2\pi f_2 n / f_s + \phi_2), \quad f_1, f_2 \in [0.1, 0.5] \text{ Hz.} \quad (2)$$

This gives you *two* evaluation regimes: (i) a controlled, ground-truth setting on synthetic wander added to real ECG, and (ii) a real-world setting on genuinely wandering records (e.g. MIT-BIH record 103 or 111, which exhibit visible baseline drift) where you do not have ground truth and must argue your results using indirect evidence.

Problems

(1) Problem Formulation and a Classical Baseline

- (a) Load record 100 from the MIT-BIH Arrhythmia Database (sampling frequency 360 Hz), extract a **30-second segment**, and add synthetic baseline wander as described above using $A_1 = 0.15$ mV, $A_2 = 0.10$ mV, $f_1 = 0.2$ Hz, and $f_2 = 0.4$ Hz. Plot the clean ECG, the synthetic baseline wander, and the corrupted ECG on the same graph with appropriate labels and legends.
- (b) Formulate baseline estimation as a supervised regression problem by clearly specifying:
- the input feature vector $\phi(n)$,
 - the target variable,
 - and the regression model to be learned.

Use **NeuroKit2** to detect the R-peaks. For each detected R-peak, construct an isoelectric anchor point by taking the median ECG amplitude in a short window ending approximately 80 ms before the R-peak. Treat these anchor amplitudes as noisy observations of the underlying baseline.

Explain why the true baseline $b[n]$ cannot be used as the regression target for a real ECG recording, and justify your choice of input features.

- (c) Implement a simple classical baseline correction method for comparison using a moving-average high-pass filter. Estimate the baseline $\hat{b}[n]$ as a long moving average of the corrupted ECG signal $x[n]$. Report the window length selected and justify your choice.
- (d) Define the evaluation metrics that will be used throughout this lab. At a minimum, report the following:
- **Mean Squared Error (MSE)** between the estimated and true baseline (available only for the synthetic experiment):

$$\text{MSE} = \frac{1}{N} \sum_{n=1}^N \left(b[n] - \hat{b}[n] \right)^2.$$

- **Baseline Reconstruction SNR (BSNR)**, which measures the quality of the estimated baseline:

$$\text{BSNR} = 10 \log_{10} \left(\frac{\sum_{n=1}^N b^2[n]}{\sum_{n=1}^N \left(b[n] - \hat{b}[n] \right)^2} \right) \text{ dB}.$$

- **Output Signal-to-Noise Ratio (Output SNR)** of the corrected ECG signal:

$$\text{SNR}_{\text{out}} = 10 \log_{10} \left(\frac{\sum_{n=1}^N s^2[n]}{\sum_{n=1}^N \left(s[n] - \hat{s}[n] \right)^2} \right) \text{ dB},$$

where

$$\hat{s}[n] = x[n] - \hat{b}[n].$$

Optionally, report the improvement in SNR as

$$\text{SNR}_{\text{imp}} = \text{SNR}_{\text{out}} - \text{SNR}_{\text{in}},$$

where

$$\text{SNR}_{\text{in}} = 10 \log_{10} \left(\frac{\sum_{n=1}^N s^2[n]}{\sum_{n=1}^N b^2[n]} \right).$$

Briefly explain why these metrics can only be computed for the synthetic experiment, where both the clean ECG $s[n]$ and the true baseline $b[n]$ are available. For the real ECG recording, discuss suitable indirect evaluation criteria, since neither the clean ECG nor the true baseline is known.

(2) Polynomial Regression for Baseline Estimation

- (a) Implement polynomial regression **from scratch**. First normalize the time index to the interval $[-1, 1]$, then construct the polynomial (Vandermonde) design matrix Φ for the chosen polynomial degree. Estimate the regression coefficients by solving the normal equations

$$(\Phi^T \Phi) \hat{w} = \Phi^T y,$$

using a suitable numerical linear solver (e.g., `numpy.linalg.solve`). Do **not** use `np.polyfit` or `sklearn.linear_model.LinearRegression` for this sub-part. Fit polynomial models of degree $p \in \{1, 3, 5, 9, 15, 25\}$ to the anchor points obtained in Problem 1(b), and evaluate the estimated baseline $\hat{b}[n]$ over the entire ECG segment.

- (b) For each polynomial degree p , compute the corrected ECG

$$\hat{s}[n] = x[n] - \hat{b}[n],$$

and plot it together with the true clean ECG $s[n]$. Identify the smallest polynomial degree at which the estimated baseline begins to noticeably distort the QRS complexes rather than simply removing the low-frequency drift. Explain why this behaviour is an indication of overfitting.

- (c) Plot both the training error (Mean Squared Error evaluated on the anchor points) and the evaluation metric(s) defined in Problem 1(d) as functions of the polynomial degree p using a single figure with dual y-axes. Discuss how these curves illustrate the bias-variance trade-off for this particular signal restoration problem.

- (d) Select the optimal polynomial degree p^* using an **ordered train/validation split**. Do not randomly shuffle the anchor points. Use the first 70% of the anchor points in time for training and the remaining 30% for validation.

For each degree p , fit the model using only the training anchors and compute the validation MSE on the held-out anchors. Plot validation MSE against p , report the selected value of p^* , and explain why an ordered split is more appropriate than a random split for this problem.

Note: The validation-selected degree p^* should be distinguished from the degree that gives the lowest full-segment MSE against the known synthetic baseline. The latter is an oracle quantity available only because this is a controlled synthetic experiment.

(3) Regularized Regression: Ridge and Lasso

- (a) Fix the polynomial degree to $p = 25$, corresponding to the deliberately over-parameterized model from Problem 2. Using the normalized polynomial features, implement **ridge regression from scratch**. Assume the first column of the design matrix corresponds to the intercept term. The intercept is not regularized. Therefore, estimate the Ridge regression coefficients by solving

$$(\Phi^T \Phi + \lambda D) \hat{w} = \Phi^T y,$$

where

$$D = \text{diag}(0, 1, \dots, 1).$$

using a suitable numerical linear solver (e.g., `numpy.linalg.solve`). Evaluate the model for 20 logarithmically spaced values of $\lambda \in [10^{-4}, 10^4]$. Plot the evaluation metric(s) defined in Problem 1(d) as functions of λ and identify the optimal regularization parameter λ^* .

- (b) Using `sklearn.linear_model.Lasso`, repeat the above experiment using ℓ_1 regularization over the same range of λ values. For both Ridge and Lasso, plot the learned coefficient vector $\hat{w}$ for: (i) a small value of λ , (ii) the selected value λ^* , and (iii) a large value of λ . Comment on how the coefficient magnitudes change as λ increases. For Lasso, use `fit_intercept=True` and provide the non-constant polynomial features to `Lasso`; the intercept should not be included as a separately regularized feature.
- (c) Compare the coefficient patterns obtained using Ridge and Lasso. Discuss the difference between coefficient *shrinkage* (Ridge) and *sparsity* (Lasso). Which method would you recommend for this problem? Support your answer by considering not only prediction accuracy but also model Interpretability and robustness when the number of polynomial basis functions becomes much larger than the number of anchor points.

- (d) Prepare a summary table comparing the best models obtained using: (i) unregularized polynomial regression (Problem 2), (ii) Ridge regression, and (iii) Lasso regression. The table should include the polynomial degree, selected value of λ , evaluation metric(s), and a brief qualitative comment on the presence or absence of QRS distortion.

Finally, plot the corrected ECG obtained using the best Ridge and best Lasso models together with the true clean ECG, and briefly comment on the differences observed.

(4) Local Polynomial Regression and Comparison with Classical Methods

- (a) A single global polynomial may not adequately model baseline drift over long ECG recordings. Implement a **local polynomial regression** approach by dividing the ECG segment into overlapping windows of approximately **4 seconds with 50% overlap**. Within each window, normalize the local time variable to the interval $[-1, 1]$ and fit a low-order polynomial of degree 3 using the anchor points that fall inside the window.

Blend the overlapping baseline estimates using a smooth cross-fade (e.g., a Hann-window cross-fade) to obtain a continuous baseline estimate $\hat{b}[n]$ over the entire ECG segment.

Justify the choice of window length and overlap, particularly in terms of the number of available anchor points within each window.

- (b) Compare the following four baseline estimation methods using the evaluation metrics defined in Problem 1(d):
- (i) Moving-average baseline correction (Problem 1),
 - (ii) Best global polynomial regression model (Problem 2),
 - (iii) Best regularized regression model (Problem 3),
 - (iv) Local polynomial regression (this problem).

Present your results as:

- a summary table of all evaluation metrics, and
- a single figure showing the corrected ECG obtained by each method, together with the true clean ECG.

Discuss the relative strengths and weaknesses of each method.

- (c) Under what recording conditions (e.g., longer recordings, rapidly varying baseline drift, or motion artifacts) would you expect local polynomial regression to outperform a single global polynomial model? Support your conclusions with at least one additional experiment rather than qualitative reasoning alone.

Finally, discuss one limitation of local polynomial regression compared with a single global polynomial model.

(5) Generalization to Real ECG Recordings

- (a) Apply the best-performing method identified from Problems 2–4 to a **real** MIT-BIH ECG record containing visible baseline wander and without any synthetic drift added. State which method you selected and justify the choice using the results obtained on the controlled synthetic experiment. Explain the distinction between selecting a model using synthetic ground truth and evaluating it on a real recording where ground truth is unavailable.

Since the true baseline $b[n]$ is unavailable, evaluate your method using at least **two indirect validation criteria**. Possible examples include:

- The estimated baseline $\hat{b}[n]$ contains predominantly low-frequency components (e.g., below 0.5 Hz).
- The corrected ECG exhibits flatter isoelectric segments.
- The amplitudes and morphology of the QRS complexes are preserved after baseline correction.

Discuss which indirect evaluation criterion you consider the most reliable and justify your answer.

- (b) (*Optional*) The hyperparameters of your model (e.g., polynomial degree p^* , regularization parameter λ^* , or local window length) were selected using synthetic baseline wander. Investigate whether these same hyperparameters generalize to the selected real ECG recording or whether re-tuning is necessary.

Discuss what this experiment reveals about deploying regression-based baseline-wander removal methods in practical biomedical signal processing, where ground-truth baseline signals are unavailable during deployment. Suggest one possible improvement that could make the model more robust across different patients or recording conditions.

Submission Instructions

- (a) Create a **private GitHub repository** named `ecm402-lab1-<rollnumber>`. Use Git throughout the development process. Make **meaningful, incremental commits** corresponding to major milestones (e.g., data loading, polynomial regression, ridge regression, evaluation, report writing). A single large commit immediately before submission is considered poor software engineering practice and may be penalized.
- (b) The repository must contain, at a minimum, the following structure:

```
ecm402-lab1-<rollnumber>/

    README.md
    requirements.txt
    report.pdf

    src/
        data.py
        regression.py
        regularization.py
        evaluate.py
        main.py

    results/
        figures/
        tables/

    notebooks/      # Optional
    data/            # Optional (small sample files only)
```

- (c) The `README.md` file should include:
- Objective of the assignment.
 - Software requirements and installation instructions.
 - How to reproduce all reported results (e.g., `python src/main.py`).
 - Brief summary of the main findings.
- (d) The `requirements.txt` file should specify the Python package versions required to reproduce your results. Do **not** upload the full MIT-BIH dataset; instead, provide instructions or scripts to download/load the dataset.
- (e) For parts explicitly marked “*from scratch*”, do **not** use `np.polyfit`, `sklearn.linear_model.LinearRegression`, `sklearn.linear_model.Ridge`, or equivalent implementations. You may use `scikit-learn` to verify your implementation, and you may use `sklearn.linear_model.Lasso` for Problem 3.
- (f) At the submission deadline, create a ZIP archive of your repository (`git archive` is preferred) excluding unnecessary files such as `.git`, `--pycache--`, virtual environments, and temporary files. Submit the ZIP file in the Google classroom.